

House Price Prediction

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
url = "https://raw.githubusercontent.com/nunnarilabs/ml/master/house/kc_house_data.csv"
df = pd.read_csv(url)
```

```
df.head()
```

	id	date	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	...	grade	s
0	7129300520	20141013T000000	221900.0	3	1.00	1180	5650	1.0	0	0	...	7	
1	6414100192	20141209T000000	538000.0	3	2.25	2570	7242	2.0	0	0	...	7	
2	5631500400	20150225T000000	180000.0	2	1.00	770	10000	1.0	0	0	...	6	
3	2487200875	20141209T000000	604000.0	4	3.00	1960	5000	1.0	0	0	...	7	
4	1954400510	20150218T000000	510000.0	3	2.00	1680	8080	1.0	0	0	...	8	

```
5 rows × 21 columns
```

```
df.shape
df.columns
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21613 entries, 0 to 21612
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  -
0   id                     21613 non-null  int64
1   date                  21613 non-null  object
2   price                 21613 non-null  float64
3   bedrooms              21613 non-null  int64
4   bathrooms             21613 non-null  float64
5   sqft_living           21613 non-null  int64
6   sqft_lot              21613 non-null  int64
7   floors                21613 non-null  float64
8   waterfront            21613 non-null  int64
9   view                  21613 non-null  int64
10  condition             21613 non-null  int64
11  grade                 21613 non-null  int64
12  sqft_above            21613 non-null  int64
13  sqft_basement         21613 non-null  int64
14  yr_built              21613 non-null  int64
15  yr_renovated          21613 non-null  int64
16  zipcode               21613 non-null  int64
17  lat                   21613 non-null  float64
18  long                  21613 non-null  float64
19  sqft_living15         21613 non-null  int64
20  sqft_lot15            21613 non-null  int64
dtypes: float64(5), int64(15), object(1)
memory usage: 3.5+ MB
```

```
df.isnull().sum()
```

	0
id	0
date	0
price	0
bedrooms	0
bathrooms	0
sqft_living	0
sqft_lot	0
floors	0
waterfront	0
view	0
condition	0
grade	0
sqft_above	0
sqft_basement	0
yr_built	0
yr_renovated	0
zipcode	0
lat	0
long	0
sqft_living15	0
sqft_lot15	0

dtype: int64

```
df = df.dropna()
```

```
df.isnull().sum()
```

```

    0
    id      0
    date     0
    price    0
    bedrooms 0
    bathrooms 0
    sqft_living 0
    sqft_lot 0
    floors    0
    waterfront 0
    view      0
    condition 0
    grade     0
    sqft_above 0
    sqft_basement 0
    yr_built  0
    yr_renovated 0
    zipcode   0
    lat       0
    long      0
    sqft_living15 0
    sqft_lot15 0

```

dtype: int64

df.describe()

	id	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view
count	2.161300e+04	2.161300e+04	21613.000000	21613.000000	21613.000000	2.161300e+04	21613.000000	21613.000000	21613.000000
mean	4.580302e+09	5.400881e+05	3.370842	2.114757	2079.899736	1.510697e+04	1.494309	0.007542	0.234
std	2.876566e+09	3.671272e+05	0.930062	0.770163	918.440897	4.142051e+04	0.539989	0.086517	0.766
min	1.000102e+06	7.500000e+04	0.000000	0.000000	290.000000	5.200000e+02	1.000000	0.000000	0.000
25%	2.123049e+09	3.219500e+05	3.000000	1.750000	1427.000000	5.040000e+03	1.000000	0.000000	0.000
50%	3.904930e+09	4.500000e+05	3.000000	2.250000	1910.000000	7.618000e+03	1.500000	0.000000	0.000
75%	7.308900e+09	6.450000e+05	4.000000	2.500000	2550.000000	1.068800e+04	2.000000	0.000000	0.000
max	9.900000e+09	7.700000e+06	33.000000	8.000000	13540.000000	1.651359e+06	3.500000	1.000000	4.000

```

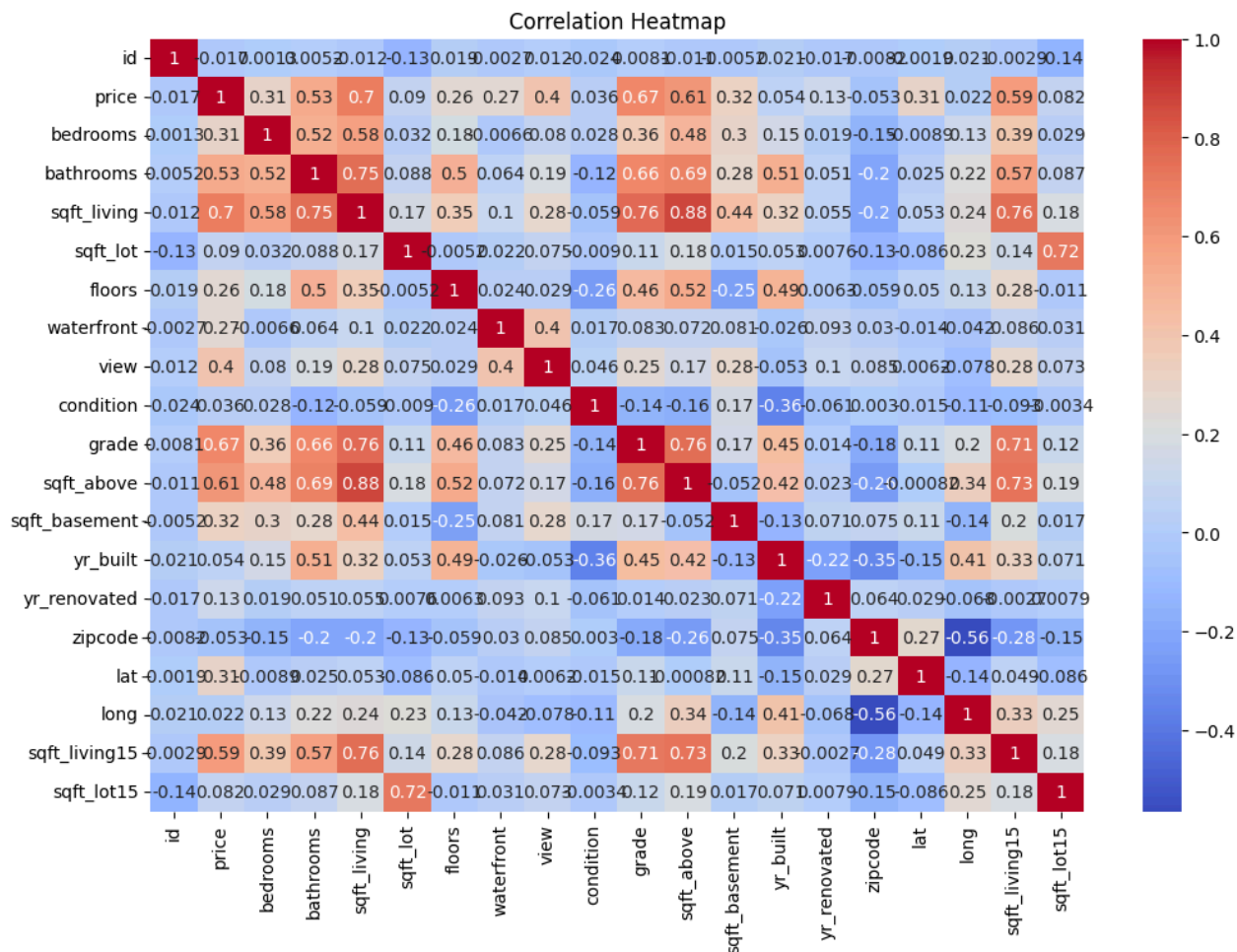
corr = df.corr(numeric_only=True)
corr

```

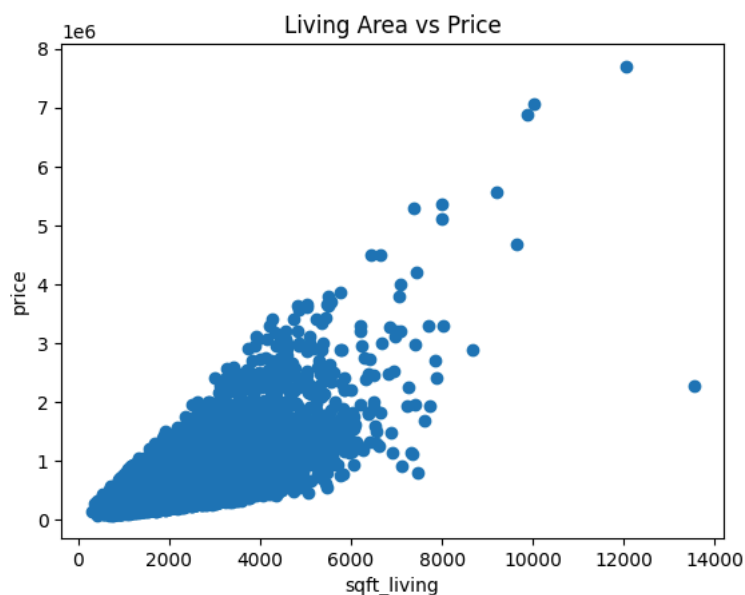
	id	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	condition	
id	1.000000	-0.016762	0.001286	0.005160	-0.012258	-0.132109	0.018525	-0.002721	0.011592	-0.023783	0.
price	-0.016762	1.000000	0.308350	0.525138	0.702035	0.089661	0.256794	0.266369	0.397293	0.036362	0.
bedrooms	0.001286	0.308350	1.000000	0.515884	0.576671	0.031703	0.175429	-0.006582	0.079532	0.028472	0.
bathrooms	0.005160	0.525138	0.515884	1.000000	0.754665	0.087740	0.500653	0.063744	0.187737	-0.124982	0.
sqft_living	-0.012258	0.702035	0.576671	0.754665	1.000000	0.172826	0.353949	0.103818	0.284611	-0.058753	0.
sqft_lot	-0.132109	0.089661	0.031703	0.087740	0.172826	1.000000	-0.005201	0.021604	0.074710	-0.008958	0.
floors	0.018525	0.256794	0.175429	0.500653	0.353949	-0.005201	1.000000	0.023698	0.029444	-0.263768	0.
waterfront	-0.002721	0.266369	-0.006582	0.063744	0.103818	0.021604	0.023698	1.000000	0.401857	0.016653	0.
view	0.011592	0.397293	0.079532	0.187737	0.284611	0.074710	0.029444	0.401857	1.000000	0.045990	0.
condition	-0.023783	0.036362	0.028472	-0.124982	-0.058753	-0.008958	-0.263768	0.016653	0.045990	1.000000	-0.
grade	0.008130	0.667434	0.356967	0.664983	0.762704	0.113621	0.458183	0.082775	0.251321	-0.144674	1.
sqft_above	-0.010842	0.605567	0.477600	0.685342	0.876597	0.183512	0.523885	0.072075	0.167649	-0.158214	0.
sqft_basement	-0.005151	0.323816	0.303093	0.283770	0.435043	0.015286	-0.245705	0.080588	0.276947	0.174105	0.
yr_built	0.021380	0.054012	0.154178	0.506019	0.318049	0.053080	0.489319	-0.026161	-0.053440	-0.361417	0.
yr_renovated	-0.016907	0.126434	0.018841	0.050739	0.055363	0.007644	0.006338	0.092885	0.103917	-0.060618	0.
zipcode	-0.008224	-0.053203	-0.152668	-0.203866	-0.199430	-0.129574	-0.059121	0.030285	0.084827	0.003026	-0.
lat	-0.001891	0.307003	-0.008931	0.024573	0.052529	-0.085683	0.049614	-0.014274	0.006157	-0.014941	0.
long	0.020799	0.021626	0.129473	0.223042	0.240223	0.229521	0.125419	-0.041910	-0.078400	-0.106500	0.
sqft_living15	-0.002901	0.585379	0.391638	0.568634	0.756420	0.144608	0.279885	0.086463	0.280439	-0.092824	0.
sqft_lot15	-0.138798	0.082447	0.029244	0.087175	0.183286	0.718557	-0.011269	0.030703	0.072575	-0.003406	0.

Next steps: [Generate code with corr](#) [New interactive sheet](#)

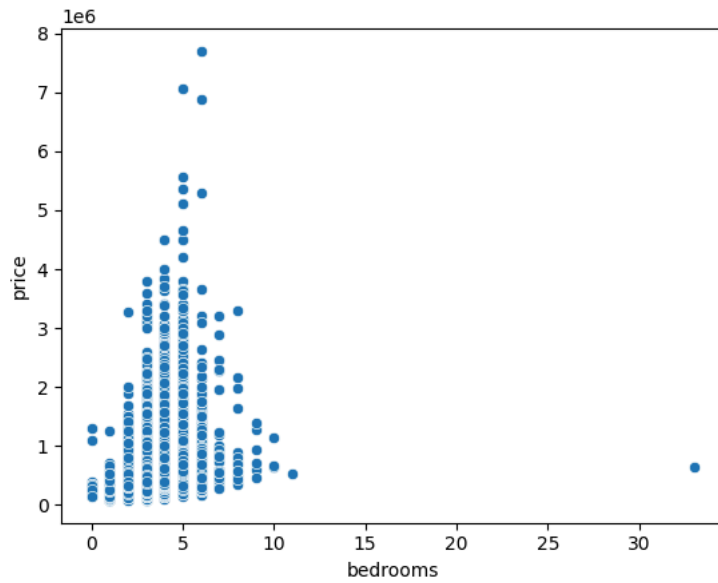
```
plt.figure(figsize=(12,8))
sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()
```



```
plt.scatter(df['sqft_living'], df['price'])
plt.xlabel("sqft_living")
plt.ylabel("price")
plt.title("Living Area vs Price")
plt.show()
```



```
sns.scatterplot(x='bedrooms', y='price', data=df)
plt.show()
```



Titanic Data Cleaning

```
df = sns.load_dataset('titanic')
df.head()
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Southampton	no	False
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	Cherbourg	yes	False
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Southampton	yes	True
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Southampton	yes	False
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Southampton	no	True

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
df['age'] = df['age'].fillna(df['age'].mean())
df['embarked'] = df['embarked'].fillna(df['embarked'].mode()[0])
```

```
df.drop('deck', axis=1, inplace=True)
```

```
df.rename(columns={
    'pclass': 'Passenger_Class',
    'sibsp': 'Siblings_Spouses',
    'parch': 'Parents_Children'
}, inplace=True)
```

```
df.columns
```

```
Index(['survived', 'Passenger_Class', 'sex', 'age', 'Siblings_Spouses',
      'Parents_Children', 'fare', 'embarked', 'class', 'who', 'adult_male',
      'embark_town', 'alive', 'alone'],
      dtype='object')
```

```
df['age'] = df['age'].astype(int)
df['survived'] = df['survived'].astype(bool)
df['Passenger_Class'] = df['Passenger_Class'].astype('category')
```

```
df.dtypes
```

```

0
survived    bool
Passenger_Class  category
sex         object
age         int64
Siblings_Spouses  int64
Parents_Children  int64
fare        float64
embarked    object
class       category
who         object
adult_male  bool
embark_town object
alive       object
alone       bool

```

dtype: object

Car Dataset Analysis

```

import pandas as pd

url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/mpg.csv"
df = pd.read_csv(url)

df.head()

```

	mpg	cylinders	displacement	horsepower	weight	acceleration	model_year	origin	name
0	18.0	8	307.0	130.0	3504	12.0	70	usa	chevrolet chevelle malibu
1	15.0	8	350.0	165.0	3693	11.5	70	usa	buick skylark 320
2	18.0	8	318.0	150.0	3436	11.0	70	usa	plymouth satellite
3	16.0	8	304.0	150.0	3433	12.0	70	usa	amc rebel sst
4	17.0	8	302.0	140.0	3449	10.5	70	usa	ford torino

Next steps: [Generate code with df](#) [New interactive sheet](#)

```

df['origin'].value_counts()

df['cylinders'].value_counts()

df['origin'].value_counts(normalize=True)

```

	proportion
usa	0.625628
japan	0.198492
europa	0.175879

dtype: float64

```

df.groupby('origin')['mpg'].mean()

df.groupby('cylinders')['horsepower'].mean()

df.groupby('cylinders')['mpg'].agg(['min', 'max', 'mean'])

```

	min	max	mean	
cylinders				
3	18.0	23.7	20.550000	
4	18.0	46.6	29.286765	
5	20.3	36.4	27.366667	
6	15.0	38.0	19.985714	
8	9.0	26.6	14.963107	

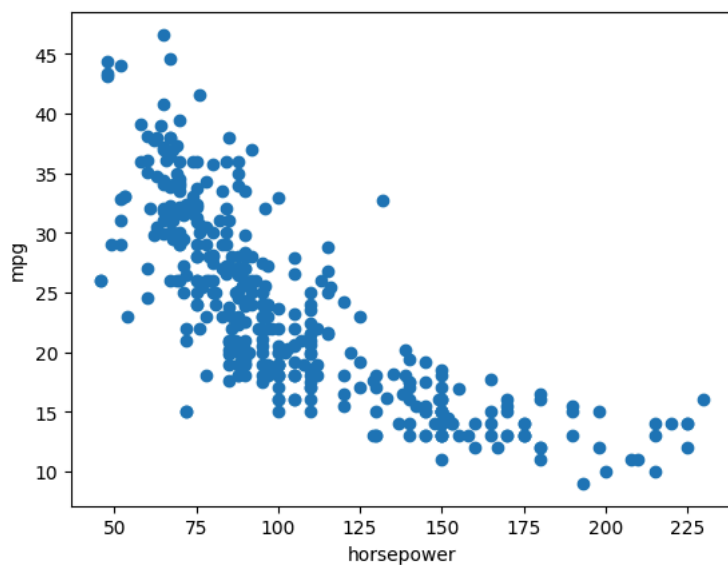
```
corr = df.corr(numeric_only=True)
corr

corr['mpg'].sort_values(ascending=False)
```

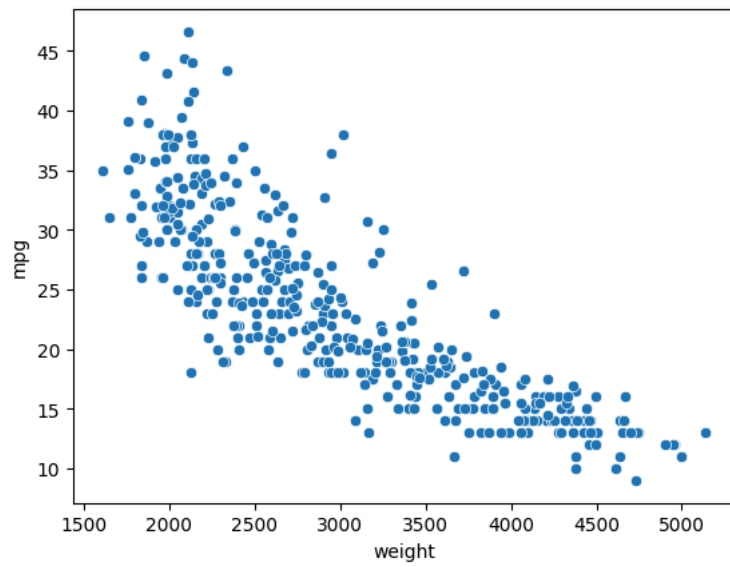
	mpg
mpg	1.000000
model_year	0.579267
acceleration	0.420289
cylinders	-0.775396
horsepower	-0.778427
displacement	-0.804203
weight	-0.831741

dtype: float64

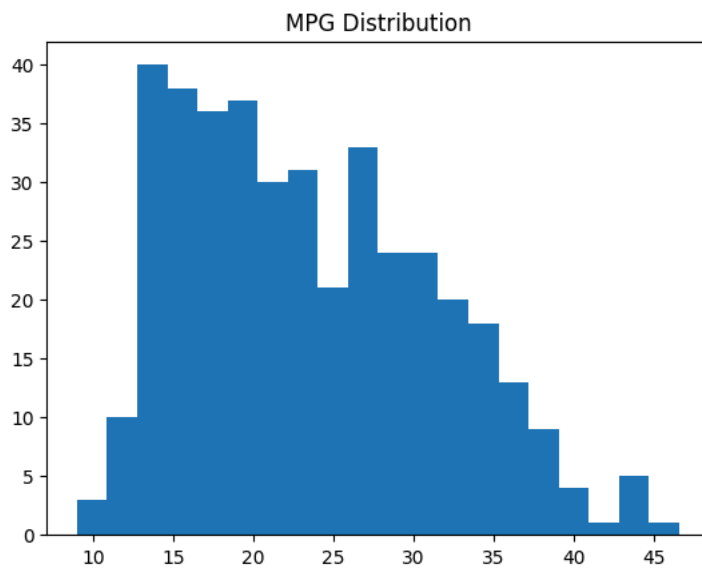
```
plt.scatter(df['horsepower'], df['mpg'])
plt.xlabel("horsepower")
plt.ylabel("mpg")
plt.show()
```



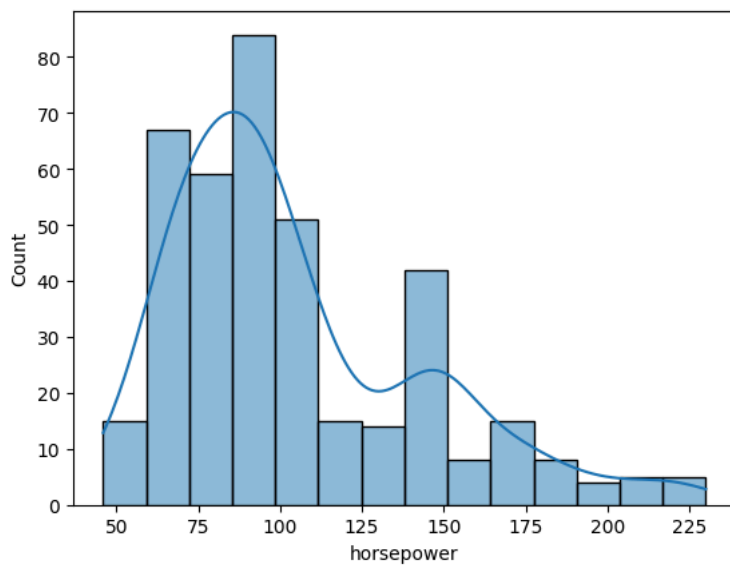
```
sns.scatterplot(x='weight', y='mpg', data=df)
plt.show()
```

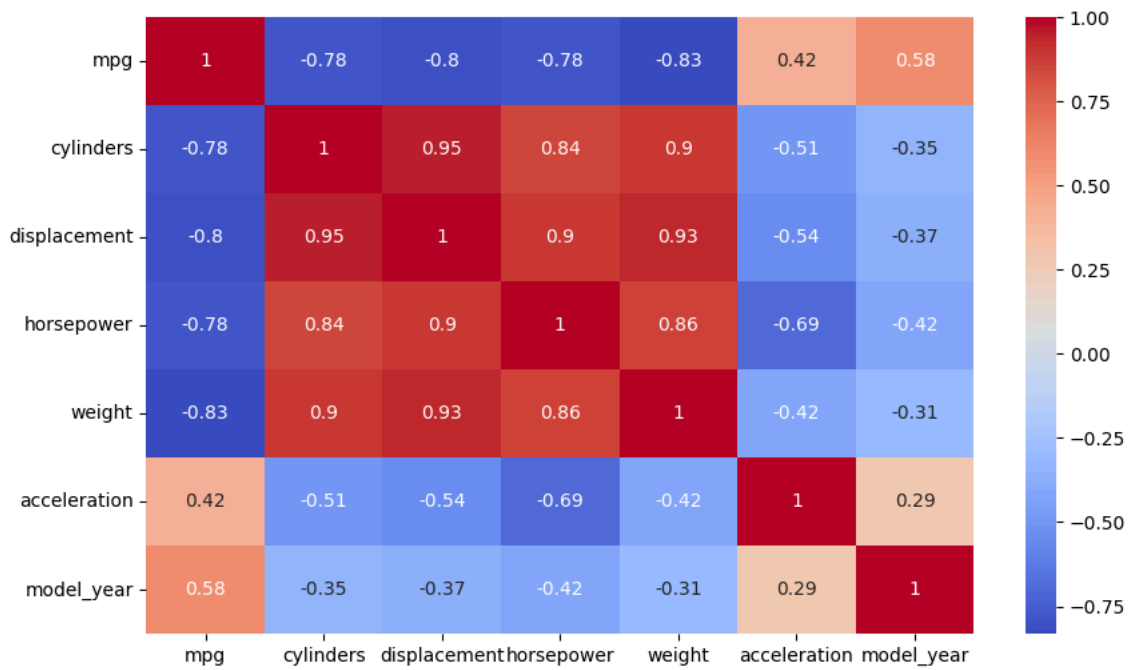
```
plt.hist(df['mpg'], bins=20)  
plt.title("MPG Distribution")  
plt.show()
```



```
sns.histplot(df['horsepower'], kde=True)  
plt.show()
```



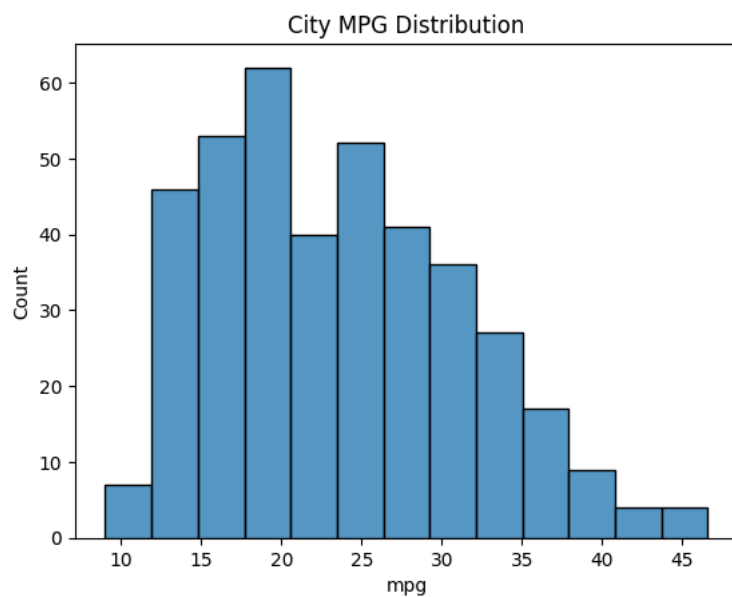
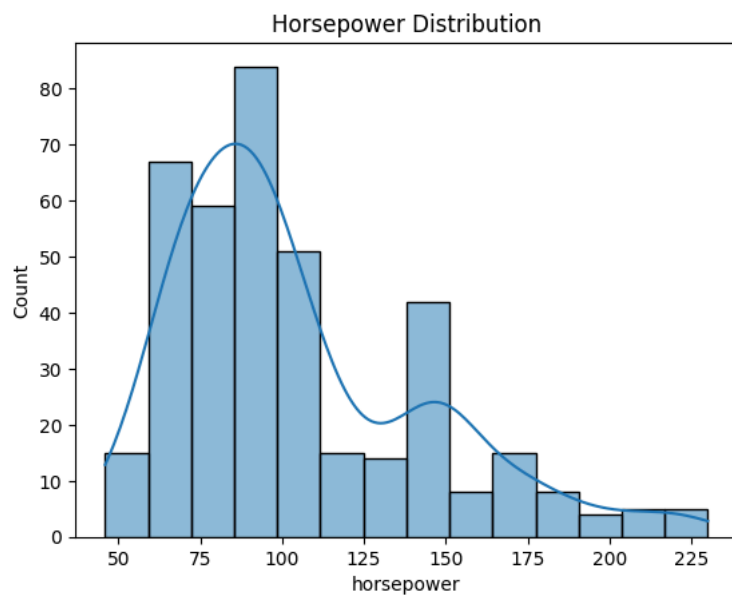
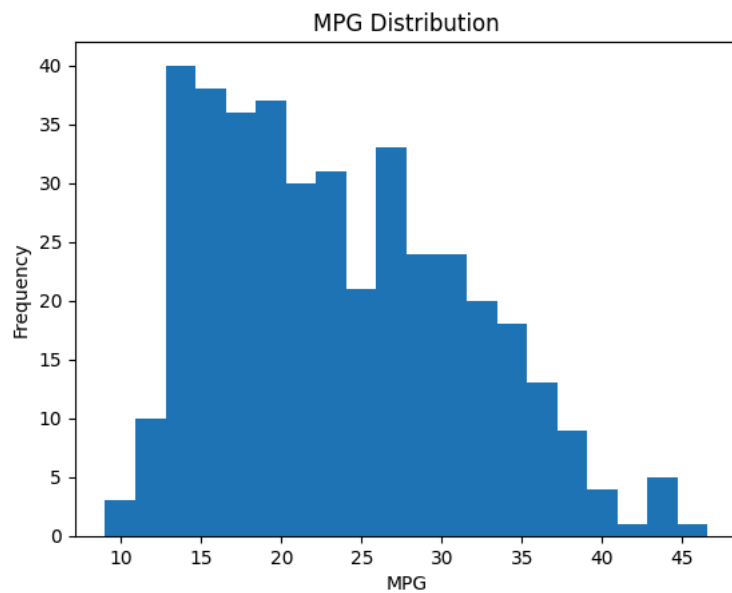
```
plt.figure(figsize=(10,6))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm')
plt.show()
```



```
plt.hist(df['mpg'], bins=20)
plt.title("MPG Distribution")
plt.xlabel("MPG")
plt.ylabel("Frequency")
plt.show()

sns.histplot(df['horsepower'], kde=True)
plt.title("Horsepower Distribution")
plt.show()

sns.histplot(df['mpg'])
plt.title("City MPG Distribution")
plt.show()
```



```
import pandas as pd

url = "https://raw.githubusercontent.com/plotly/datasets/master/imports-85.csv"
df_auto = pd.read_csv(url)

df_auto.head()
```

	symboling	normalized-losses	make	fuel-type	aspiration	num-of-doors	body-style	drive-wheels	engine-location	wheel-base	...	engine-size	fuel-system	bore
0	3	NaN	alfa-romero	gas	std	two	convertible	rwd	front	88.6	...	130	mpfi	3.47
1	3	NaN	alfa-romero	gas	std	two	convertible	rwd	front	88.6	...	130	mpfi	3.47
2	1	NaN	alfa-romero	gas	std	two	hatchback	rwd	front	94.5	...	152	mpfi	2.68
3	2	164.0	audi	gas	std	four	sedan	fwd	front	99.8	...	109	mpfi	3.19
4	2	164.0	audi	gas	std	four	sedan	4wd	front	99.4	...	136	mpfi	3.19

5 rows × 26 columns

```
df_auto.columns
df_auto.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 205 entries, 0 to 204
Data columns (total 26 columns):
#   Column                Non-Null Count  Dtype
---  -
0   symboling              205 non-null   int64
1   normalized-losses      164 non-null   float64
2   make                   205 non-null   object
3   fuel-type              205 non-null   object
4   aspiration              205 non-null   object
5   num-of-doors           203 non-null   object
6   body-style             205 non-null   object
7   drive-wheels           205 non-null   object
8   engine-location        205 non-null   object
9   wheel-base             205 non-null   float64
10  length                 205 non-null   float64
11  width                  205 non-null   float64
12  height                 205 non-null   float64
13  curb-weight            205 non-null   int64
14  engine-type            205 non-null   object
15  num-of-cylinders       205 non-null   int64
16  engine-size            205 non-null   int64
17  fuel-system            205 non-null   object
18  bore                   201 non-null   float64
19  stroke                 201 non-null   float64
20  compression-ratio      205 non-null   float64
21  horsepower             203 non-null   float64
22  peak-rpm               203 non-null   float64
23  city-mpg               205 non-null   int64
24  highway-mpg            205 non-null   int64
25  price                  201 non-null   float64
dtypes: float64(11), int64(6), object(9)
memory usage: 41.8+ KB
```

```
df_auto['make'].value_counts()

df_auto['body-style'].value_counts()

df_auto['drive-wheels'].value_counts(normalize=True)

df_auto['fuel-system'].value_counts()
```

	count
fuel-system	
mpfi	94
2bbl	66
idi	20
1bbl	11
spdi	9
4bbl	3
mfi	1
spfi	1

```
dtype: int64
```

```
# Average price by make
df_auto.groupby('make')['price'].mean()

# Average horsepower by body-style
df_auto.groupby('body-style')['horsepower'].mean()

# Average city-mpg by drive-wheels
df_auto.groupby('drive-wheels')['city-mpg'].mean()

# Min, max, mean price by body-style
df_auto.groupby('body-style')['price'].agg(['min', 'max', 'mean'])

# Make + body-style combination
df_auto.groupby(['make', 'body-style'])['price'].mean().sort_values(ascending=False)
```


		price
make	body-style	
porsche	convertible	37028.000000
mercedes-benz	hardtop	36788.000000
	convertible	35056.000000
jaguar	sedan	34600.000000
porsche	hardtop	33278.000000
mercedes-benz	sedan	33074.000000
	wagon	28248.000000

```
corr = df_auto.corr(numeric_only=True)
```

```
corr
```

	volvo	sedan	normalized	18726.875000	wheel-	length	width	height	curb-	num-of-	engine-	bore	st
	toyota	symboling	convertible	17669.000000	base				weight	cylinders	size		
	symboling	1.000000		0.528667	-0.531954	-0.357612	-0.232919	-0.541038	-0.227691	-0.113129	-0.105790	-0.134205	-0.00
	normalized-	0.528667	hatchback	16500.000000	0.074362	0.023220	0.105073	-0.432335	0.119893	0.175380	0.167365	-0.036167	0.06
	losses	-0.531954		-0.074362	1.000000	0.874587	0.795144	0.589435	0.776386	0.339507	0.569329	0.490378	0.16
	wheel-base	-0.357612	wagon	16290.000000	0.874587	1.000000	0.841118	0.491029	0.877728	0.430672	0.683360	0.607480	0.12
	length	-0.232919		0.105073	0.795144	0.841118	1.000000	0.279210	0.867032	0.545007	0.735433	0.559204	0.18
	width	-0.232919	sedan	15433.333333	0.589435	0.491029	0.279210	1.000000	0.295572	-0.013995	0.067149	0.176195	-0.05
	height	-0.541038		-0.432335	0.589435	0.491029	0.279210	1.000000	0.295572	-0.013995	0.067149	0.176195	-0.05
	curb-weight	-0.227691	hatchback	15013.333333	0.776386	0.877728	0.867032	0.295572	1.000000	0.609727	0.850594	0.649045	0.16
	num-of-	-0.113129	convertible	14997.500000	0.339507	0.430672	0.545007	-0.013995	0.609727	1.000000	0.846031	0.243553	0.00
	cylinders	-0.113129		0.175380	0.339507	0.430672	0.545007	-0.013995	0.609727	1.000000	0.846031	0.243553	0.00
	alfa-romero	0.167365		0.569329	0.683360	0.735433	0.067149	0.850594	0.846031	1.000000	0.594090	0.20	
	engine-size	-0.105790	wagon	12290.000000	0.490378	0.607480	0.559204	0.176195	0.649045	0.243553	0.594090	1.000000	-0.05
	bore	-0.134205		0.065627	0.490378	0.607480	0.559204	0.176195	0.649045	0.243553	0.594090	1.000000	-0.05
	volkswagen	-0.008965		0.161477	0.129739	0.182956	-0.056999	0.168929	0.008578	0.206675	-0.055909	1.00	
	stroke	-0.008965	sedan	10462.666667	0.249786	0.158414	0.181129	0.261214	0.151362	-0.020002	0.028971	0.005203	0.18
	compression-	-0.150000		0.490378	0.607480	0.559204	0.176195	0.649045	0.243553	0.594090	1.000000	-0.05	
	mazda	0.071622	hatchback	10085.000000	0.352297	0.555003	0.642482	-0.110711	0.751034	0.691633	0.810773	0.577273	0.09
	horsepower	0.071622		0.264597	-0.361052	-0.287325	-0.219957	-0.322272	-0.266306	-0.124434	-0.244618	-0.264269	-0.07
	mazda	0.274573	hardtop	9762.333333	-0.361052	-0.287325	-0.219957	-0.322272	-0.266306	-0.124434	-0.244618	-0.264269	-0.07
	peak-rpm	-0.035823		-0.258502	-0.470414	-0.670909	-0.642704	-0.048640	-0.757414	-0.445837	-0.653658	-0.594584	-0.04
	city-mpg	-0.035823	sedan	9945.000000	-0.544082	-0.704662	-0.677218	-0.107358	-0.797465	-0.466666	-0.677470	-0.594572	-0.04
	honda	0.034606		-0.210768	-0.544082	-0.704662	-0.677218	-0.107358	-0.797465	-0.466666	-0.677470	-0.594572	-0.04
	highway-mpg	0.034606	hatchback	9895.000000	-0.544082	-0.704662	-0.677218	-0.107358	-0.797465	-0.466666	-0.677470	-0.594572	-0.04
	price	-0.082391		0.203254	0.584642	0.690628	0.751265	0.135486	0.834415	0.708645	0.872335	0.543436	0.08
	renault	-0.082391		0.203254	0.584642	0.690628	0.751265	0.135486	0.834415	0.708645	0.872335	0.543436	0.08
	toyota	0.034606	wagon	9836.000000	-0.544082	-0.704662	-0.677218	-0.107358	-0.797465	-0.466666	-0.677470	-0.594572	-0.04
ct steps:		Generate code with corr											
		hardtop											

Next steps:

[Generate code with corr](#)

[New interactive sheet](#)

```
corr.unstack().sort_values(ascending=False)
```

toyota	hatchback	9616.000000
mitsubishi	hatchback	9597.888889
toyota	sedan	9542.200000
subaru	wagon	9342.000000
renault	wagon	9295.000000
subaru	sedan	9070.600000
plymouth	wagon	8921.000000
dodge	wagon	8921.000000
nissan	sedan	8604.555556
mitsubishi	sedan	8434.000000
nissan	hardtop	8249.000000
plymouth	hatchback	8130.500000
dodge	hatchback	7819.800000
	sedan	7619.666667

honda wagon 7295.000000 0			
corr['price'].sort_values(ascending=False)			
highway-mpg	highway-mpg	1.000000	
normalized-losses	normalized-losses	1.000000	
price	price	1.000000	6591.333333
engine-size	peak-rpm	0.872335	1.000000
peak-rpm	peak-rpm	1.000000	
curb-weight	curb-weight	0.834415	5723.000000
horsepower	horsepower	0.810533	-0.770908
width	width	0.751265	-0.797465
num-of-cylinders	num-of-cylinders	0.708645	-0.797465
length	length	0.690628	-0.803620
wheel-base	wheel-base	0.584642	-0.803620
bore	bore	0.543436	
normalized-losses	normalized-losses	0.203254	
dtype: float64			